```
$ typeprof lib/aaagmnr.rb
# TypeProf 0.21.3

# Classes
module Aaagmnr
  VERSION: String

  class Finder
    @signatures: Hash[String, Array[String]]

    def self.from_file: (untyped file_name) -> Finder
    def initialize: (Array[String] dictionary_words) -> void
    def lookup: (untyped word) -> Array[String]?
    def signature_of: (String word) -> String
  end

  class Options
    DEFAULT_DICTIONARY: String

    attr_reader dictionary: String?
    attr_reader words_to_find: untyped
    def initialize: (untyped argv) -> void

    private
    def parse: (untyped argv) -> bot
  end

  class Runner
    @options: bot

    def initialize: (untyped argv) -> void
    def run: -> untyped
  end

  class Error < StandardError
  end
end
```

This is quite similar to the RBS file that we created by hand, with a few changes:

- We forgot a couple of constants, like `DEFAULT_DICTIONARY` and `Error`.
- In quite a few cases, TypeProf can't determine a type and puts in `untyped`. We know that `Finder#lookup` takes a String and returns an array of strings, but TypeProf returns `def lookup: (untyped word) -> Array[String]?`, meaning that it can't infer a type for the parameter, and it's also assuming that `nil` is a potential output (which, looking at the code as written, is correct—that method can return `nil` if the word being looked up is not in the dictionary).

What you get from TypeProf, then, is a mix of items that we know about the code that TypeProf can't figure out, and items that TypeProf can figure out but that we, the developers, didn't necessarily see. This makes TypeProf a useful way to start with RBS, but not necessarily the completed goal.

As an alternative to running TypeProf from the command line, an experimental plugin for Visual Studio Code (https://marketplace.visualstudio.com/items?itemName=mame.ruby-typeprof) will generate method type signatures as you write code.

In order to generate these type signatures, TypeProf executes your Ruby code…kind of. The phrase the documentation uses is "abstractly executes," which means that it walks through the code paths knowing the types of variables, but not their values.